

Simple Quantum Algorithms II

QML

Contents

1	Overview	3
2	The Quantum Fourier Transform	3
2.1	Classical DFT Review	3
2.2	QFT Definition	3
2.3	QFT for Small Cases	3
2.4	QFT Circuit	4
2.5	Inverse QFT	4
3	Quantum Phase Estimation	5
3.1	The Problem	5
3.2	The Circuit	5
3.3	How It Works	5
3.4	Worked Example	6
4	Shor's Algorithm	6
4.1	The Factoring Problem	6
4.2	Reduction to Order Finding	7
4.3	Quantum Order Finding	7
4.4	Complexity	8
5	Grover's Search Algorithm	8
5.1	The Problem	8
5.2	The Algorithm	8
5.3	The Oracle	8
5.4	The Diffusion Operator	9
5.5	Geometric Interpretation	9
5.6	Multiple Solutions	10
5.7	Grover's Optimality	10
5.8	Grover Worked Example	10
5.9	Amplitude Amplification	11

6	Quantum Speedups: Quadratic vs Exponential	11
7	Summary	12

1 Overview

this lecture covers the heavy hitters: quantum fourier transform (QFT), quantum phase estimation (QPE), shor's algorithm, and grover's search. these are the algorithms that actually matter for the real world — shor's for cryptography, grover's for search, and QPE as a subroutine in basically everything.

for QML specifically, QPE shows up in quantum kernel estimation, and grover's search connects to amplitude amplification techniques used in quantum classifiers. the variational structure of QAOA (lecture 10) can be seen as a “poor man's” version of adiabatic evolution, which connects to QPE.

2 The Quantum Fourier Transform

2.1 Classical DFT Review

the discrete fourier transform (DFT) maps a vector $\mathbf{x} = (x_0, x_1, \dots, x_{N-1})$ to $\mathbf{y} = (y_0, y_1, \dots, y_{N-1})$:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \omega^{jk}, \quad \omega = e^{2\pi i/N}$$

classically, the FFT computes this in $O(N \log N)$ operations. quantumly, we can do it in $O(\log^2 N)$ gates — exponentially faster. but there's a catch we'll get to.

2.2 QFT Definition

Definition 2.1. the quantum fourier transform on $N = 2^n$ basis states:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i jk/N} |k\rangle$$

equivalently, the QFT can be written in product form:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{l=1}^n \left(|0\rangle + e^{2\pi i j/2^l} |1\rangle \right)$$

Intuition. the product form is key for building the circuit. it says the QFT maps each computational basis state to a product of single-qubit states, where each qubit gets a diff phase. this means we can build the QFT using single-qubit rotations and controlled rotations — no need for complicated multi-qubit entangling operations.

2.3 QFT for Small Cases

Example 2.1. QFT on 1 qubit ($N = 2$):

$$\text{QFT}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & e^{i\pi} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = H$$

the 1-qubit QFT is just the hadamard gate. makes sense — the hadamard is the fourier transform over $\mathbb{Z}_2$.

Example 2.2. QFT on 2 qubits ($N = 4$):

$$\text{QFT}_4 = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$$

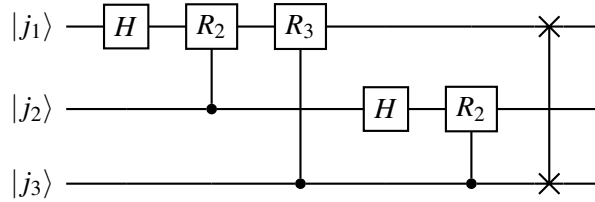
2.4 QFT Circuit

the QFT circuit for n qubits uses hadamard gates and controlled phase rotations:

Definition 2.2. the controlled phase rotation gate:

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{2\pi i/2^k} \end{pmatrix}$$

for $n = 3$ qubits:



where R_k is the phase gate $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$, and the SWAP at the end reverses the qubit order.

Key Point. QFT circuit complexity:

- uses n hadamard gates and $n(n-1)/2$ controlled rotations
- total: $O(n^2)$ gates for n qubits
- since $N = 2^n$, this is $O(\log^2 N)$ gates
- compared to classical FFT: $O(N \log N)$ operations
- this is an exponential speedup in gate count

but heres the catch: u cant efficiently read out all the amplitudes after the QFT. measurement only gives u one sample. so the QFT is useful as a subroutine (inside QPE, shor's, etc.), not as a standalone algorithm for computing fourier transforms.

2.5 Inverse QFT

the inverse QFT is just the adjoint circuit — run the QFT circuit backwards with all rotations negated:

$$\text{QFT}^{-1} |k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{-2\pi i j k / N} |j\rangle$$

in circuit form: reverse the gate order and replace R_k with $R_k^\dagger = R_k^{-1}$.

3 Quantum Phase Estimation

3.1 The Problem

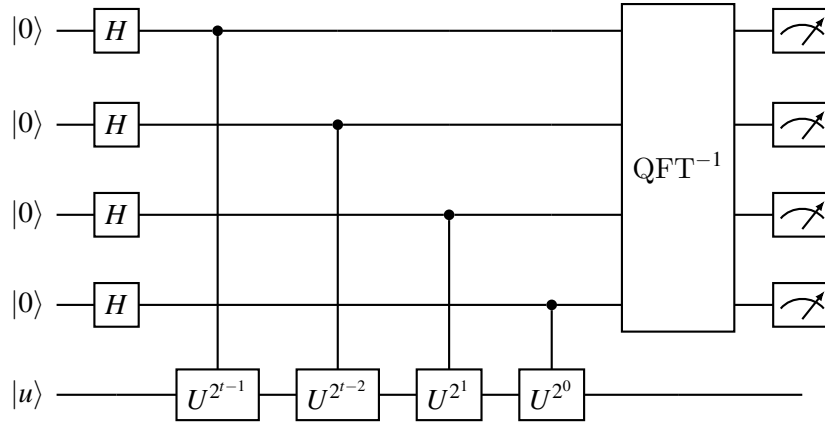
given a unitary U and one of its eigenstates $|u\rangle$ with $U|u\rangle = e^{2\pi i\phi}|u\rangle$, estimate the phase $\phi \in [0, 1)$.

this is one of the most important quantum subroutines. it shows up in:

- shor's factoring algorithm
- quantum chemistry (ground state energy estimation)
- quantum counting (estimating the number of solutions for grover)
- HHL algorithm for linear systems
- various QML algorithms

3.2 The Circuit

QPE uses two registers: a counting register of t qubits (determines precision) and an eigenstate register holding $|u\rangle$.



3.3 How It Works

step 1: hadamards create a uniform superposition in the counting register.

step 2: controlled- U^{2^j} operations. the j -th counting qubit controls U^{2^j} applied to $|u\rangle$. by phase kickback:

$$|+\rangle|u\rangle \xrightarrow{C-U^{2^j}} \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i \cdot 2^j \phi} |1\rangle)|u\rangle$$

after all controlled operations, the counting register is:

$$\frac{1}{\sqrt{2^t}} \sum_{k=0}^{2^t-1} e^{2\pi i k \phi} |k\rangle$$

this is exactly the QFT of $|2^t \phi\rangle$ (if $2^t \phi$ is an integer).

step 3: inverse QFT undoes the fourier transform:

$$\text{QFT}^{-1} \left(\frac{1}{\sqrt{2^t}} \sum_{k=0}^{2^t-1} e^{2\pi i k \phi} |k\rangle \right) = |\tilde{\phi}\rangle$$

where $\tilde{\phi}$ is the t -bit binary representation of ϕ .

step 4: measure the counting register to get $\tilde{\phi}$.

Key Point. QPE precision:

- if $\phi = m/2^t$ for some integer m (“exact case”): QPE gives the exact answer with probability 1
- if ϕ is not exactly representable in t bits: QPE gives the closest t -bit approximation with high probability
- using t qubits gives t bits of precision: $|\tilde{\phi} - \phi| \leq 2^{-t}$
- success probability $\geq 1 - \epsilon$ requires $t = n + \lceil \log(2 + 1/(2\epsilon)) \rceil$ qubits where n is the desired bits of precision

3.4 Worked Example

Example 3.1. estimate the phase of $T|1\rangle$, where $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$.

$T|1\rangle = e^{i\pi/4}|1\rangle$, so $e^{2\pi i \phi} = e^{i\pi/4}$, giving $\phi = 1/8 = 0.001$ in binary.

use $t = 3$ counting qubits for exact representation.

after controlled- T^{2^j} operations and phase kickback, the counting register is:

$$\frac{1}{\sqrt{8}} \sum_{k=0}^7 e^{2\pi i k/8} |k\rangle$$

after inverse QFT: $|001\rangle$ (which is 1 in binary, representing $\phi = 1/8$).

measure: get 001, so $\phi = 1/2^3 = 1/8$. correct!

4 Shor’s Algorithm

4.1 The Factoring Problem

given a composite integer N , find a nontrivial factor.

why it matters: RSA encryption relies on the assumption that factoring large numbers is hard. shor’s algorithm factors in polynomial time on a quantum computer, which would break RSA.

classical: best known algorithm is the general number field sieve with complexity $\exp(O(n^{1/3}(\log n)^{2/3}))$ where $n = \log N$. sub-exponential but super-polynomial.

quantum: shor’s algorithm runs in $O(n^2 \log n \log \log n)$ — polynomial.

4.2 Reduction to Order Finding

shor's key insight: factoring reduces to order finding.

Definition 4.1. the order of a modulo N is the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$.

reduction:

1. pick a random $a < N$. if $\gcd(a, N) > 1$, we found a factor (done).
2. find the order r of $a \bmod N$.
3. if r is even, compute $\gcd(a^{r/2} \pm 1, N)$. with probability $\geq 1/2$, one of these is a nontrivial factor.
4. if r is odd or $a^{r/2} \equiv -1 \pmod{N}$, try a different a .

Example 4.1. factor $N = 15$, pick $a = 7$.

$$7^1 = 7, 7^2 = 49 \equiv 4, 7^3 \equiv 28 \equiv 13, 7^4 \equiv 91 \equiv 1 \pmod{15}$$

order $r = 4$ (even, good).

$$\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$$

$$\gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$$

factors: $15 = 3 \times 5$.

4.3 Quantum Order Finding

the quantum part uses QPE with the unitary:

$$U_a |x\rangle = |ax \bmod N\rangle$$

the eigenstates of U_a r:

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle$$

with eigenvalues $e^{2\pi i s / r}$ for $s = 0, 1, \dots, r-1$.

QPE estimates $\phi = s/r$, and from the fraction s/r we can extract r using the continued fractions algorithm.

Key Point. shor's algorithm outline:

1. classical preprocessing: pick random a , check $\gcd(a, N) = 1$
2. quantum: run QPE on U_a with $|1\rangle$ as the eigenstate register (which is a superposition of the $|u_s\rangle$)
3. measure: get an estimate of s/r for random s
4. classical: use continued fractions to extract r from the estimate
5. classical: compute $\gcd(a^{r/2} \pm 1, N)$ to get factors

the quantum part (QPE) is the only step that needs a quantum computer. everything else is classical.

4.4 Complexity

- QPE needs $O(\log N)$ qubits for the counting register
- modular exponentiation ($U_a^{2^j}$) needs $O(\log^2 N)$ gates per controlled operation
- total gate count: $O(\log^3 N)$ or $O(\log^2 N \log \log N)$ with fast multiplication
- polynomial in the number of digits of N

Intuition. shor's algorithm is not just abt factoring. its abt period finding in general. the quantum fourier transform reveals periodic structure in the eigenvalues of a unitary. this is the continuous-group generalization of simons algorithm (which found periodicity over $\mathbb{Z}_2^n$). shor's does it over $\mathbb{Z}_N$.

5 Grover's Search Algorithm

5.1 The Problem

given an oracle $f : \{0, 1\}^n \rightarrow \{0, 1\}$ that "marks" M solutions out of $N = 2^n$ possibilities (i.e., $f(x) = 1$ for exactly M values of x), find a marked item.

classically: $O(N/M)$ queries (linear search). for a single solution ($M = 1$): $O(N)$ queries.

quantumly: $O(\sqrt{N/M})$ queries. for $M = 1$: $O(\sqrt{N})$ queries.

this is a quadratic speedup. not exponential like shor's, but applicable to a much wider class of problems.

5.2 The Algorithm

Definition 5.1. Grover's algorithm for a single solution ($M = 1$):

1. initialize: $|s\rangle = H^{\otimes n} |0^n\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle$
2. repeat $\lfloor \frac{\pi}{4} \sqrt{N} \rfloor$ times:
 - (a) apply the oracle: $O_f |x\rangle = (-1)^{f(x)} |x\rangle$ (flip phase of solution)
 - (b) apply the diffusion operator: $D = 2|s\rangle\langle s| - I$ (reflect about the mean)
3. measure

5.3 The Oracle

the oracle marks the solution by flipping its phase:

$$O_f |x\rangle = \begin{cases} -|x\rangle & \text{if } f(x) = 1 \\ |x\rangle & \text{if } f(x) = 0 \end{cases}$$

in matrix form for $N = 4$ with solution $|w\rangle = |10\rangle$:

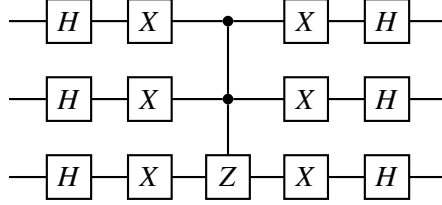
$$O_f = I - 2|w\rangle\langle w| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

5.4 The Diffusion Operator

the diffusion operator reflects the state about the uniform superposition $|s\rangle$:

$$D = 2|s\rangle\langle s| - I$$

circuit implementation:



more precisely: $D = H^{\otimes n}(2|0^n\rangle\langle 0^n| - I)H^{\otimes n}$

the inner part $(2|0^n\rangle\langle 0^n| - I)$ flips the phase of everything except $|0^n\rangle$. this can be implemented as: X on all qubits, multi-controlled Z, X on all qubits.

5.5 Geometric Interpretation

the best way to understand grover's algorithm is geometrically, in a 2D plane spanned by $|w\rangle$ (the solution) and $|w^\perp\rangle$ (everything else).

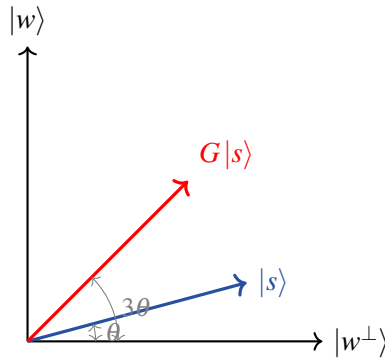
Definition 5.2. define:

$$|w^\perp\rangle = \frac{1}{\sqrt{N-1}} \sum_{x:f(x)=0} |x\rangle$$

then the initial state is:

$$|s\rangle = \sin\theta |w\rangle + \cos\theta |w^\perp\rangle$$

where $\sin\theta = \frac{1}{\sqrt{N}}$ (for $M=1$), so $\theta \approx \frac{1}{\sqrt{N}}$ for large N .



each grover iteration (oracle + diffusion) is a rotation by 2θ in this 2D plane:

$$G|s\rangle = \sin(3\theta) |w\rangle + \cos(3\theta) |w^\perp\rangle$$

after k iterations:

$$G^k |s\rangle = \sin((2k+1)\theta) |w\rangle + \cos((2k+1)\theta) |w^\perp\rangle$$

we want the amplitude of $|w\rangle$ to be close to 1, which happens when $(2k+1)\theta \approx \pi/2$:

$$k \approx \frac{\pi}{4\theta} \approx \frac{\pi}{4}\sqrt{N}$$

Key Point. grover's algorithm is a rotation in a 2D plane. each iteration rotates by $2\theta \approx 2/\sqrt{N}$ towards the solution. after $\pi/(4\theta) \approx (\pi/4)\sqrt{N}$ rotations, the state is nearly aligned with $|w\rangle$, and measurement gives the solution with high probability.

if u iterate too many times, u “overshoot” and the probability decreases. this is unlike classical search where more iterations always help. u need to know (or estimate) the right number of iterations.

5.6 Multiple Solutions

for M solutions out of N :

$$\theta = \arcsin \sqrt{M/N}$$

number of iterations: $k \approx \frac{\pi}{4}\sqrt{N/M}$

Example 5.1. $N = 1024$, $M = 4$ solutions.

$$\theta = \arcsin(2/32) = \arcsin(1/16) \approx 0.0625$$

$$k \approx \frac{\pi}{4}\sqrt{256} = \frac{\pi}{4} \cdot 16 \approx 12.6$$

so approximately 13 iterations.

5.7 Grover's Optimality

Key Point. grover's algorithm is optimal for unstructured search. the BBBV theorem (Bennett, Bernstein, Brassard, Vazirani, 1997) proves that any quantum algorithm for unstructured search requires $\Omega(\sqrt{N})$ queries. u cant do better than grover's.

this means quantum computers give a quadratic speedup for unstructured search, not exponential. the exponential speedups (like shor's) require structure in the problem (e.g., periodicity, group structure).

5.8 Grover Worked Example

Example 5.2. $N = 4$ (2 qubits), searching for $|w\rangle = |11\rangle$.

$$\theta = \arcsin(1/2) = \pi/6. \text{ number of iterations: } (2k+1)\pi/6 = \pi/2 \Rightarrow k = 1.$$

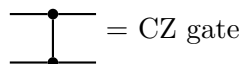
initialize:

$$|s\rangle = H^{\otimes 2}|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

oracle (flip phase of $|11\rangle$):

$$O_f |s\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$$

the oracle circuit for marking $|11\rangle$:



diffusion operator $D = 2|s\rangle\langle s| - I$:

$$|s\rangle\langle s| = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

$$D = 2|s\rangle\langle s| - I = \frac{1}{2} \begin{pmatrix} -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix}$$

$$D \cdot O_f |s\rangle = \frac{1}{2} \begin{pmatrix} -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix} \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ -1 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |11\rangle$$

after one iteration, the state is exactly $|11\rangle$. measure and get the solution with probability 1.

5.9 Amplitude Amplification

grover's algorithm is a special case of the more general amplitude amplification technique.

Definition 5.3. given any quantum algorithm $\mathcal{A}$ that produces the correct answer with probability p , amplitude amplification can boost the success probability to near 1 using $O(1/\sqrt{p})$ repetitions of $\mathcal{A}$ and $\mathcal{A}^{-1}$.

Intuition. amplitude amplification is to grover's search what the abstract fourier transform is to the FFT — its the general principle behind the specific algorithm. in grover's, the “algorithm” $\mathcal{A}$ is just creating a uniform superposition (hadamard), and the “correct answer” is the marked item. but amplitude amplification works with any starting algorithm, making it useful in many QML contexts where u have a quantum subroutine that succeeds with some probability and u want to boost it.

6 Quantum Speedups: Quadratic vs Exponential

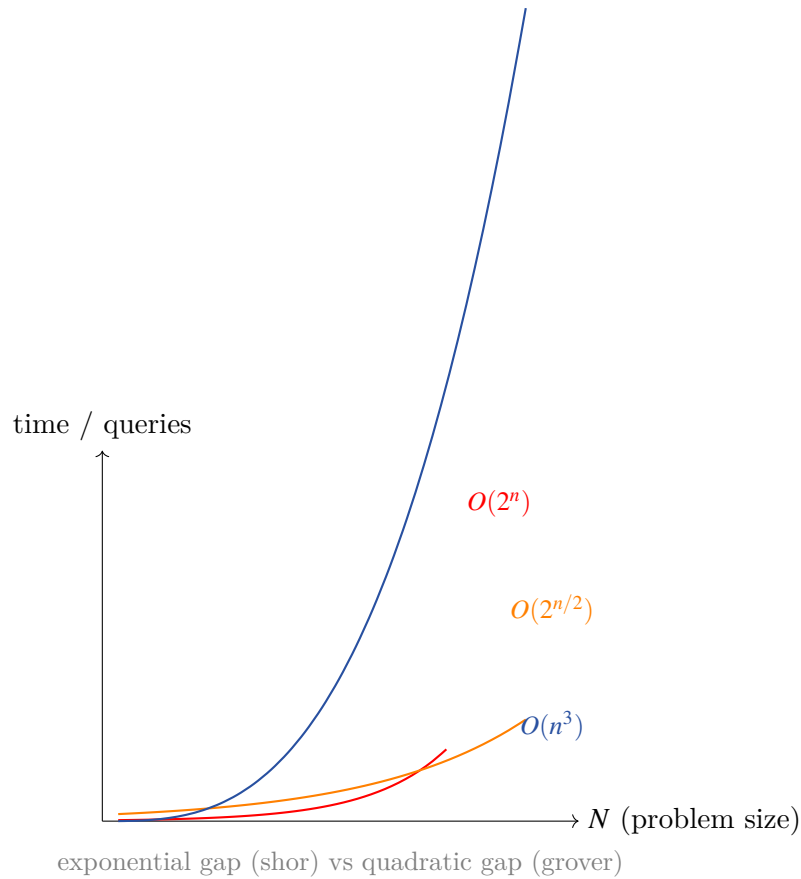
Key Point. its important to understand the diff types of quantum speedups:

exponential speedups (like shor's): require specific mathematical structure in the problem. not generic — u cant get an exponential speedup for arbitrary problems.

quadratic speedups (like grover's): more broadly applicable but less dramatic. $\sqrt{N}$ instead of N is nice but not world-changing for most practical problem sizes.

polynomial speedups: various algorithms offer speedups that r polynomial but not quite quadratic. often these come with caveats (need QRAM, specific input models, etc.).

for QML: most quantum speedups for machine learning r either quadratic (grover-type) or require specific assumptions abt data access. exponential speedups r rare and usually rely on the data having special structure (like low-rank matrices). this is why the QML advantage question is so nuanced.



7 Summary

Key Point. the big algorithms:

1. QFT: $O(\log^2 N)$ gate complexity, fourier transform over $\mathbb{Z}_N$. cant be used standalone bcs of measurement limitations, but is an essential subroutine
2. QPE: estimates eigenvalue phases using QFT. foundation for shor's, HHL, quantum chemistry, and many QML algorithms
3. Shor's: polynomial-time factoring via QPE + order finding. exponential speedup. breaks RSA. uses the structure of modular arithmetic
4. Grover's: $O(\sqrt{N})$ unstructured search. quadratic speedup, provably optimal. generalizes to amplitude amplification

connection to QML: QPE connects to quantum kernel estimation and quantum principal component analysis. amplitude amplification can boost the success probability of quantum classifiers. the QFT shows up in feature maps for quantum machine learning. shor's is less directly relevant to QML but demonstrates the kind of structure quantum computers exploit.